BÁO CÁO BÀI GIỮA KỲ

Chuyên đề phát triển phần mềm

Đề tài: Web Live Stream (WebRTC + Signaling WebSocket)

Thông tin	Nội dung
Họ và tên	Huỳnh Minh Tiến
MSSV	1721031693
Môn học	Chuyên đề phát triển phần mềm
Repository mã nguồn	https://github.com/HuynhMinhTien-1721031693/CHUYENDEPHATTRIENPHANMEM_GIUAKY
Video demo (YouTube)	https://youtu.be/3nTMk1xiUGs

I. Lời mở đầu

Em thực hiện đề tài **Web Live Stream** trong khuôn khổ môn **Chuyên đề phát triển phần mềm** (bài giữa kỳ). Đây là một trang web cho phép một người **phát** hình ảnh và âm thanh từ camera và micro trên trình duyệt, và người khác **xem trực tiếp** bằng mã phòng hoặc link chia sẻ.

Qua đề tài, em cố gắng ôn luyện và thể hiện các phần môn học đã học: làm việc với **Git/GitHub**, xây **frontend** bằng React, xây **backend** bằng Node.js, và viết **kiểm thử tự động**. Phần "xương sống" kỹ thuật là **WebRTC** (truyền media trực tiếp giữa hai trình duyệt) và **WebSocket** trên server chỉ để **trao đổi tín hiệu** (signaling), không phải để chuyển tiếp toàn bộ video như các nền tảng stream truyền thống.

Báo cáo được trình bày theo thứ tự: giới thiệu sản phẩm → công nghệ và cách làm → hướng dẫn chạy thử và demo → kết quả, hạn chế, hướng phát triển → tài khoản quản trị.

II. Mô tả sản phẩm (ứng dụng làm gì, ai dùng)

II.1. Vấn đề cần giải quyết

Trong phạm vi bài giữa kỳ, em cần một sản phẩm **chạy được trên trình duyệt**, có **tương tác thời gian thực** (phát – xem), và có **mã nguồn + tài liệu + video demo** để nộp.

II.2. Giải pháp tổng quan

- **Người phát (host):** vào trang phát, cho phép trình duyệt dùng camera và micro, hệ thống tạo **mã phòng** và **đường link** để chia sẻ.
- **Người xem (viewer):** mở link hoặc nhập mã phòng ở trang chủ; sau khi kết nối ổn định, viewer nhận **luồng video/audio** từ host.

- **Server:** dùng **WebSocket** để chuyển các thông điệp kỹ thuật (offer, answer, ICE candidate) giúp hai trình duyệt "bắt tay" với nhau. Phần hình ảnh và âm thanh chính đi **trực tiếp giữa host và viewer** (peer-to-peer) sau khi thiết lập xong.

II.3. Giới hạn hợp lý của bản demo

- Mã phòng được **chuẩn hóa** (độ dài và ký tự cho phép) để tránh nhập sai và đồng bộ với backend.
- Số viewer tối đa mỗi phòng có **giới hạn** (theo cấu hình demo, ví dụ 20 người) để tránh quá tải trong môi trường thử nghiệm.

III. Công nghệ và kiến trúc

III.1. Danh sách công nghệ

Lớp	Công nghệ dùng trong đề tài
Giao diện người dùng	React 18, Vite, React Router
Media trên trình duyệt	WebRTC (`getUserMedia`, `RTCPeerConnection`), STUN công khai (Google)
Máy chủ	Node.js, Express, thư viện `ws` (WebSocket)
API bổ trợ	REST: kiểm tra phòng (`/api/room/:id/exists`), kiểm tra hoạt động (`/api/health` có `version`)
Bảo mật cơ bản	Giới hạn kích thước body JSON; header `X-Content-Type-Options: nosniff`
Kiểm thử	Jest + Supertest (backend); Vitest + Testing Library (frontend)
Quản lý mã nguồn	Git, GitHub

III.2. Kiến trúc ngắn gọn (bằng lời)

Có thể hình dung **hai luồng**:

1. **HTTP/REST** — viewer (và đôi khi các thao tác khác) gọi API để **biết phòng có tồn tại** trước khi mở kết nối WebSocket.
2. **WebSocket** — host và viewer **trao đổi tín hiệu** để WebRTC thiết lập đường truyền. Sau đó, **media không đi qua Node** theo kiểu "chuyển tiếp toàn bộ video" mà đi **P2P** giữa các peer.

III.3. Sơ đồ kiến trúc và luồng WebRTC

Cách xem sơ đồ: các khối `mermaid` hiển thị trên GitHub hoặc VS Code (Markdown Preview Mermaid). Để chèn vào Word/PowerPoint, xuất PNG tại mermaid.live (dán mã bên dưới).

III.3.1. Kiến trúc tổng quan (REST, WebSocket, P2P, STUN)

```
flowchart TB
  subgraph BH["Trình duyệt — Host"]
    HR["React HostLive"]
    HPC["WebRTC: getUserMedia + RTCPeerConnection"]
  end
  subgraph BV["Trình duyệt — Viewer"]
    VR["React WatchLive"]
    VPC["WebRTC: RTCPeerConnection"]
  end
  subgraph NODE["Máy chủ Node.js :3001"]
    REST["Express — REST"]
    WS["WebSocket /live-signal — liveRooms relay SDP/ICE"]
  end
  STUN[("STUN công khai\nstun.l.google.com:19302")]
  HR -->|"GET /api/room/.../exists, /api/health"| REST
  VR -->|"GET /api/room/.../exists"| REST
  HR <-->|"JSON signaling"| WS
  VR <-->|"JSON signaling"| WS
  HPC <-->|"RTP/SRTP trực tiếp (P2P)"| VPC
  HPC -.->|"ICE candidate"| STUN
  VPC -.->|"ICE candidate"| STUN
```

III.3.2. Trình tự signaling (một viewer vào phòng)

```
sequenceDiagram
  participant H as Host
  participant Sig as "Node — WebSocket /live-signal"
  participant V as Viewer
  H->>Sig: host-start
  Sig->>H: room-ready (roomId)
  V->>Sig: viewer-join (roomId)
  Sig->>V: joined (viewerId)
  Sig->>H: viewer-joined (viewerId)
  Note over H: addTrack, createOffer
  H->>Sig: offer + viewerId + sdp
  Sig->>V: offer + sdp
  Note over V: setRemoteDescription, createAnswer
  V->>Sig: answer + sdp
  Sig->>H: answer + viewerId + sdp
  par ICE song song
    H->>Sig: ice-host (viewerId, candidate)
    Sig->>V: ice (candidate)
    V->>Sig: ice-viewer (candidate)
    Sig->>H: ice (viewerId, candidate)
  end
  Note over H,V: RTP/SRTP — media trực tiếp P2P sau khi SDP/ICE xong
```

III.3.3. Bản tóm tắt in / PDF (khi Mermaid không được render)

```
[Host]  -- REST -->  [Express :3001]  <-- REST --  [Viewer]
[Host]  <==== WebSocket JSON: offer, answer, ICE ====>  [Viewer]
[Host]  ========== RTP/SRTP (P2P, không qua Node) ======>  [Viewer]
[Host]  .... STUN (ICE) ....                    [Viewer]
```

III.4. Một vài xử lý kỹ thuật đáng nói

- **ICE buffer:** đôi khi gói ICE candidate đến **trước** khi hoàn tất thiết lập mô tả phiên (SDP); code dùng **bộ đệm** rồi **flush** sau khi sẵn sàng — giúp giảm lỗi kết nối.
- **Chuẩn hóa mã phòng** thống nhất giữa frontend và backend.
- **Tách REST và WebSocket:** minh họa rõ vai trò từng kênh trong ứng dụng thực tế.

III.5. Quy mô mã nguồn (LOC) và ước lượng Story point

LOC (Lines of Code): đo bằng script `scripts/bao-cao-gk/count-loc.mjs` — đếm **dòng vật lý** (có tính dòng trống và comment) trong `frontend/src` (tách file `*.test.*`) và `backend/src`, `backend/tests`. Kết quả mới nhất lưu tại `scripts/bao-cao-gk/loc-snapshot.json` (cập nhật mỗi lần chạy `node count-loc.mjs`). Số liệu này được đưa vào slide PowerPoint (bảng + biểu đồ tròn).

Story point: ước lượng theo thang **Fibonacci** cho từng nhóm epic (giao diện + SPA Task + live hub, WebRTC, auth/quản trị, backend signaling, test & báo cáo). Story point phản ánh **độ phức tạp và effort tương đối**, **không** suy ra tự động từ LOC — dùng khi mô tả quy mô công việc hoặc lập kế hoạch sprint (xem bảng trong `BaoCao_GK.pptx`).

IV. Cách chạy dự án, kiểm thử và hướng dẫn demo

IV.1. Chuẩn bị môi trường

1. Cài **Node.js** (phiên bản tương thích với dự án).
2. Clone hoặc tải mã nguồn từ GitHub về máy.
3. Mở **hai cửa sổ terminal** (hoặc hai tab) trong thư mục dự án.

IV.2. Chạy backend (cửa sổ thứ nhất)

```
cd backend
npm install
npm run dev
```

Đợi terminal báo server chạy (thường cổng **3001**).

IV.3. Chạy frontend (cửa sổ thứ hai)

```
cd frontend
npm install
npm run dev
```

Mở trình duyệt tại địa chỉ dev (thường `http://localhost:5173`).

IV.4. Kiểm thử tự động (nên quay trong video demo)

Backend:

```
cd backend
npm test
```

Frontend:

```
cd frontend
npm test
```

(Trên Windows PowerShell, chạy từng lệnh trong đúng thư mục.)

IV.5. Demo luồng host → viewer

1. **Tab hoặc máy thứ nhất:** vào trang chủ → chọn **Tạo phòng phát**.
2. Khi trình duyệt hỏi, **cho phép** camera và micro.
3. Chờ hệ thống hiện **mã phòng** và **link** → bấm **Sao chép** link (hoặc ghi mã phòng).
4. **Tab ẩn danh hoặc máy thứ hai:** dán link hoặc vào trang chủ, nhập **mã phòng**, bấm **Vào xem**.
5. Quan sát: phía viewer có **hình và tiếng** từ host sau vài giây (tùy mạng).

IV.6. Lưu ý khi demo

- Ưu tiên **cùng mạng Wi-Fi** cho host và viewer lần đầu.
- Tắt **VPN** nếu có xung đột.
- Kiểm tra **firewall** nếu không kết nối được.
- Một số loại NAT khó có thể cần **TURN** (ngoài phạm vi bản demo hiện tại).

V. Kết quả đạt được

- Ứng dụng chạy **end-to-end**: host phát, viewer xem được trên **tab khác hoặc máy khác** khi điều kiện mạng phù hợp.
- **Test tự động** backend và frontend chạy được bằng `npm test`.
- Có **mã nguồn** trên GitHub và **báo cáo** (Word/PDF/PowerPoint) kèm **link video** theo yêu cầu giảng viên.

VI. Hạn chế

- Demo chủ yếu dùng **STUN**; một số mạng cần **TURN** để relay media thì có thể **không xem được** dù code đúng.
- Triển khai lên domain thật cần **HTTPS** và **wss://** cho WebSocket.

- Quy mô lớn (rất nhiều viewer) thường cần kiến trúc **SFU/MCU**, khác với demo P2P hiện tại.

VII. Hướng phát triển

- Tích hợp **TURN** khi cần thiết.
- Mở rộng **xác thực**, quản lý phòng, ghi log.
- Cải thiện **giao diện**, trải nghiệm mobile, và giám sát chất lượng luồng.

VIII. Kết luận

Đề tài đã hoàn thành một vòng đời sản phẩm nhỏ nhưng đủ các phần: **frontend**, **backend**, **realtime signaling**, **kiểm thử**, **tài liệu** và **video minh họa**. Em rút ra được kinh nghiệm về WebRTC, WebSocket, và cách phối hợp nhiều công cụ trong một dự án full-stack ngắn.

IX. Tài khoản quản trị và phân quyền (Quản lý hệ thống / Admin phòng live)

Ứng dụng có **hai vai trò quản trị tách biệt** (không thay thế lẫn nhau).

IX.1. Quản lý hệ thống (System manager)

- **Mục đích:** chỉnh các **thiết lập toàn site** (tên hiển thị, thông báo chào mừng, chế độ bảo trì).
- **Không** dùng để quản lý phòng phát, host hay viewer.
- **Sau khi đăng nhập:** vào **Quản lý hệ thống** (ví dụ đường dẫn `/system`).

IX.2. Admin phòng live (Admin)

- **Mục đích: quản lý phòng live** — xem danh sách phòng, có thể **đóng phòng**, xem **danh sách người dùng** đã tạo tài khoản qua form **Đăng ký** trên web.
- **Không** dùng để chỉnh cấu hình hệ thống như mục IX.1.
- **Sau khi đăng nhập:** vào **Admin phòng live** (ví dụ `/admin`).

IX.3. Hai quyền quản trị không được tạo qua giao diện web

- Trang **Đăng ký** trên website **chỉ** tạo tài khoản vai trò **người dùng thường** (user); backend **không** gán `sysmanager` hay `admin` qua API đăng ký công khai.
- **Quản lý hệ thống** và **Admin phòng live không** có chức năng "tự đăng ký" trên web; đây là yêu cầu an toàn để **không** lộ quyền cao qua form công khai.
- Hai tài khoản này **chỉ được khởi tạo phía máy chủ** khi chạy backend: đọc biến môi trường **`SEED_*`** trong file **`backend/.env`** (mẫu trong **`backend/.env.example`**) và ghi vào dữ liệu cục bộ nếu email chưa tồn tại. Sau đó người quản trị **đăng nhập** bằng email/mật khẩu đã cấu hình — **không** phải tạo mới qua trang Đăng ký.

IX.4. Các bước khởi tạo trên máy chủ (môi trường demo)

1. Trong thư mục **`backend`**, sao chép **`.env.example`** thành **`.env`**.
2. Điền **`SEED_SYSMANAGER_EMAIL`**, **`SEED_SYSMANAGER_PASSWORD`**, **`SEED_ADMIN_EMAIL`**, **`SEED_ADMIN_PASSWORD`** (email/mật khẩu dùng cho hai vai trò quản trị).
3. Chạy backend (`npm run dev` hoặc `npm start`). Mỗi lần khởi động, hai email `SEED_*` trong **`.env`** được **tạo mới hoặc đồng bộ** trong `backend/data/app-data.json` (đúng vai trò **Quản lý hệ thống** / **Admin phòng live** và mật khẩu local), kể cả khi trước đó đã đăng ký web trùng email.
4. Mở trang **Đăng nhập** trên web và đăng nhập bằng **email + mật khẩu** đã khai báo trong `.env` — đây là **đăng nhập**, không phải **đăng ký** tạo tài khoản quản trị.

Nếu cần mật khẩu khác với `.env`, chỉnh `SEED_*_PASSWORD` rồi khởi động lại backend, hoặc sửa tay `app-data.json`.

IX.5. Bảng tài khoản và mật khẩu mẫu (theo `.env.example`)

STT	Vai trò	Email (mẫu)	Mật khẩu (mẫu)	Ghi chú
1	**Quản lý hệ thống**	`sys@local.test`	`QuanLyHT@Demo2026`	Chỉ tạo qua seed `.env`, không qua web Đăng ký.
2	**Admin phòng live**	`admin@local.test`	`AdminLive@Demo2026`	Chỉ tạo qua seed `.env`, không qua web Đăng ký.

An toàn: bảng trên phục vụ **báo cáo và demo cục bộ**; triển khai thật cần **đổi mật khẩu mạnh** và **không** đưa `.env` lên Git.

Phụ lục — Thông tin nộp bài

Mục	Nội dung
Họ và tên	Huỳnh Minh Tiến
MSSV	1721031693
Môn học	Chuyên đề phát triển phần mềm
Đề tài	Web Live Stream (WebRTC + Signaling WebSocket)
Link GitHub	https://github.com/HuynhMinhTien-1721031693/CHUYENDEPHATTRIENPHANMEM_GIUAKY
Link video YouTube	https://youtu.be/3nTMk1xiUGs